

4. Bölüm

C Programlama Diline Giriş

4.1 En Basit C Programı

C programlama dili, her programcının ihtiyacını görecektir genel amaçlı, emreden paradigmalı , yapısal programlama dilidir. Kısa ve öz talimatlar birbiri ardına verilerek yapılan programlamaya emreden paradigma (**imperative programming**) adı verilir. Yapısal programlamanın çerçevesi 2.7 başlığı altında anlatılmıştır.

Yapısal programlamada, programın başladığı yeri belirten bir **ana fonksiyon (main function)** tanımlanır. C dilinde bu fonksiyon **int main()** fonksiyonudur. Ana fonksiyonun olmadığı hiçbir yapısal program çalışmaz. En basit C programı aşağıdaki gibi yazılabilir;

```
1 int main()
2 {
3     return 0;
4 }
```

Kodu şu şekilde açıklayabiliriz;

- ◊ Birinci satırda, yapısal programlamanın ana fonksiyonu olan (**main function**) tanımlanmıştır. Başındaki **int**, fonksiyonun bir tam sayı (**integer**) geri döndüreceğini belirtir. Program çalışmaya ana fonksiyondan başlar ve bittiğinde işletim sistemine geri döner.
- ◊ C dilinde her fonksiyon ikinci satırdaki gibi süslü parantez karakteri ile başlayan ve dördüncü satırdaki gibi biten bir **kod bloğuna (code block)** sahiptir. Her fonksiyonda olan bu blok, fonksiyon bloğu (**function block**) olarak da adlandırılır.
- ◊ Ana fonksiyon içinde üçüncü satırda yazılan **return 0;** talimatı (**statement**) ana fonksiyondan sıfır geri döndürerek çıktıldığını söyleyen talimattır. Sıfır dışında bir değer işletim sistemine gönderilmesi, programın hata ile sonlandığı olarak kabul edilir. Eğer programdan çıkışta işletim sistemine 11 göndermek istersek bu talimatı **return 11;** olarak yazmalıyız.

4.2 Söz Dizim Kuralları

C dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kurallarına (syntax)** uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler

1. İngiliz alfabesindeki büyük harfler:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
2. İngiliz Alfabesindeki küçük harfler:
a b c d e f g h i j k l m n o p q r s t u v w x y z
3. Rakamlar:
0 1 2 3 4 5 6 7 8 9
4. Özel Karakterler:
< > . , _ () ; \$: % [] # ? & { } ^ ! \ + - * / = ~ ' "
5. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk (white space)** karakterleri;
 - (a) Boşluk (**space**)
 - (b) Yeni satır (**new line**)
 - (c) Satır başı (**carriage return**)
 - (d) Geri (**backspace**)
 - (e) Sekme (**tab**)

§4.2.2 Talimatlar ve Anahtar Kelimeler

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (**reserved words**) olup programı oluşturan talimatlar (**statement**) Tablo 4.1’de verilen bu kelimeler kullanılarak yazılırlar.

Talimat

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç emir kodunu (**instruction code**) temsil edebilir. Talimatlar, yapılması gerekeni kısa ve net olarak belirten mantıksal satırlardır (**logical sequence**).

auto	else	long	switch
break	enum	register	typedef
case	extern	return	union
char	float	short	unsigned
const	for	signed	void
continue	goto	sizeof	volatile
default	if	static	while
do	int	struct	double

Tablo 4.1: Anahtar Kelimeler

Anahtar Kelimelerin Yazımı

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle** yazılırlar.

Talimatlar BASIC ve FORTRAN gibi dillerde genellikle satır sonunda biter. FORTRAN dilinde yedinci sütundan yazılmaya başlanır. BASIC dilinde ise her satırın başına satır numarası yazılır. Pascal ve C dillerinde böyle bir kural söz konusu değildir. Talimat herhangi bir yerde başlar ve noktalı virgül karakteri ile biter. Aradaki beyaz boşluklar ne kadar çok olursa olsun tek bir tane varmış gibi kabul edilir. Bu nedenle kod **gelişigüzel** (**free format**) yazılır.

Talimatlar

Her talimat (**statement**), anahtar kelimeler içerecek şekilde yazılır ve **noktalı virgül karakteri ile biter**

Aşağıda gelişigüzel yazılan programların hepsi derlenebilir ve derleme sonucu aynı çalıştırılabilir dosyayı üretir. Beyaz boşlukları alınmış en kısa kod aşağıda verilmiştir:

```
int main() {return 0;}
```

Kod içerisinde beyaz boşluk karakterleri anahtar kelimeleri bölmediği sürece istenildiği kadar kullanılabilir:

```
int          main(          ) {
    return 0; }
```

Fonksiyon parantezleri içine veya fonksiyon blokları içine istenildiği kadar beyaz boşluk konulabilir:

```
int main (          )
{ return 0; }
```

Kodun güzel görünmesi şart değildir:

```
int main() {
    return          0;
}
```

Anahtar kelime argümanları arasına da beyaz boşluklar gelebilir:

```
int main()  {
return          0;
}
```

Parantezler ve fonksiyon blokları hizalı olmayabilir:

```
int main(
)  {
```

```

    return
    0;
}

```

Ancak kodun bir başkası tarafından bakımı yapılacağı göz önüne alınarak okunaklı yazılması çok önemlidir. **En doğrusu programı aşağıdaki sitilde yazmaktır;**

- ◊ Fonksiyonların gövdesi ve bazı kontrol talimatları kod bloklarından oluşur. Kod blokları için süslü parantez kullanılır.
- ◊ Süslü parantez içindeki kodlar, okunabilirliği artırmak için genellikle **4 boşluk** veya 1 **sekme (tab)** içeriden başlar. Kodu bu şekilde yazmaya **girintili (indented)** yazma adı verilir.
- ◊ Ayrıca değişkenler ve anahtar kelimeler arasında boşluk karakteri veya sekme karakteri kullanılmalıdır:

```

int main() {
    //Ana fonksiyon bloğuna ait girinti
    int sayac=0;
    if (sayac==0) { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
    if (sayac==1) // Blok başlangıcı böyle de yazılabilir
    { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
    return 0;
}

```

§4.2.3 Açıklamalar

Talimatlar (statement), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır (logical sequence)**. Programcı kodu yazarken **açıklama (comment)** yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü (//) karakteri kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama (/*) ile (*/) karakterleri arasına alınır.

Aşağıda açıklamalar eklenmiş ana fonksiyon örneği yer almaktadır;

```

/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan
C programıdır. İlhan ÖZKAN, Aralık 2024
*/
int main() // Yapısal Programlamadaki "Ana Fonksiyon"
{ //fonksiyon bloğu başlangıcı
    /* Ana Fonksiyon, programın başladığı yerdir. Ana fonksiyonu olmayan bir program çalışmaz. */

    return 0; /*fonksiyondan dönüş talimatı noktalı virgül karakteri ile bitmiştir.*/
} //fonksiyon bloğu bitişi

```

Kod İçerisindeki Açıklamalar

İstedığımız kadar açıklama yapabiliriz. Derleyici açısından bunun önemi yoktur. Çünkü derlemenin ilk aşamasında **bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.**

4.3 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi C dilinde de **veri yapıları (data structure)** ve **kontrol yapıları (control structure)** birbirinden ayrıdır. Dolayısıyla veriyi işleyecek kontrol yapısı kodlanmadan önce veriyi taşıyan veri yapıları tanımlanmalıdır.

Değişkenler (variable) veri yapılarının temel öğeleridir. Bu başlıkta anlatılan tamsayı ve kayan noktalı sayı değişkenleri, belleğin belli bir bölgesini işgal eder ve fiziki olarak bellek bitlerinden oluştuğundan ikili sayı (**binary**) olarak veri tutarlar.

C dilinde ise 2.4 başlığında anlatılan tamsayı ve kayan noktalı sayılar, **ilkel veri tipleri** (**primitive data type**) olarak adlandırılır. Bu veri tipleri için kullanılan anahtar kelimeler aşağıda verilmiştir;

1. Tamsayı olarak kullanılacak veri tipleri (**integer data type**);
 - (a) **char**: bellekte 1 bayt yani 8 bit yer kaplayan işaretli tamsayıları tutan veri tipidir.
 - (b) **short**: bellekte 2 bayt yani 16 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
 - (c) **int**: bellekte 2/4 bayt yani 16/32 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
 - (d) **long**: bellekte 4/8 bayt yani 32/64 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
2. Gerçek sayı veri tipleri (**floating point data type**);
 - (a) **float**: bellekte 4 bayt yani 32 bit yer kaplayan **tek hassasiyetli** (**single precision**) kayan noktalı gerçek sayıları tutan veri tipidir.
 - (b) **double**: bellekte 8 bayt yani 64 bit yer kaplayan **çift hassasiyetli** (**double precision**) kayan noktalı gerçek sayıları tutan veri tipidir.

void Anahtar Kelimesi

C programlama dilinde bir de **hiçbir değeri olmayan ve bellekte yer kaplamayan void** veri tipi vardır. Bu veri tipini hiçbir değer döndürmeyen fonksiyon tanımlamalarında ve dinamik bellek kullanımlarında ileride çokça göreceğiz.

Değişkenlerin veri tipine (**data type**) bağlı olarak benzersiz bir **kimlik** (**identifier**) verilerek **tanımlanması** (**definition**) gerekir.

```
/* Bu program değişken tanımlamak için oluşturulmuştur.*/
int main() {
    char yas; /* veri tipi "char" ve kimliği "yas" olan değişken tanımlandı. -128 ila +127
               ↳ arasında değer alabilir. */
    short kat; /* veri tipi "short" ve kimliği "kat" olan değişken tanımlandı. -32768 ila +32767
               ↳ arasında değer alabilir. */
    float kilo; /* veri tipi "float" ve kimliği "kilo" olan değişken tanımlandı. */
    return 0;
}
```

Yukarıda tanımlaması yapılan **yas**, **kat** ve **kilo** değişkenleri ana fonksiyon bloğu içinde tanımlandığından, tanımlanan değişkenler sadece bu blok içinde geçerlidir!

Aynı Bloktaki Değişkenler

Aynı blok içinde bir değişkene verilen kimlik, bir başka değişkene verilemez!

Bildirim (**declaration**) derleyiciye böyle bir değişken, fonksiyon veya nesne var demektir. **Tanımlama**, (**definition**) derlemenin sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

İşaretli bir tamsayı kullanmak yerine en anlamlı işaret bitini de sayıya dahil ederek tam sayıları işaretsiz olarak (**unsigned**) kullanabiliriz. Tamsayı veri tipleri olan **char**, **short**, **int** ve **long** veri tipleri (**data type**) aslında **işaretli** (**signed**) tamsayılardır. Yani önlerinde **signed** sıfatı olduğu varsayılır. Tam sayıları işaretsiz olarak kullanmak istememiz halinde **unsigned** sıfatını tamsayı veri tipinin önünde kullanırız.

```
/* Bu program işaretsiz tamsayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
    unsigned yas; /* veri tipi "unsigned int" ve kimliği "yas" değişkeni
                  tanımlandı. yas değişkeni 0 ile 255 arasında bir değer alabilir. */
    unsigned short kat; /* veri tipi "unsigned short" ve kimliği "kat" olan değişken
                        ↳ tanımlandı. kat değişkeni 0 ila 65535 arasında değer alabilen bir değişkendir. */
    unsigned int pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında değer
                                ↳ alabilen bir değişkendir. */
    return 0;
}
```

Bir de bellekte tahsis edilen yeri artıran **long** sıfatı vardır; **int** veri tipini **long** sıfatı ile tanımladığımızda **long int** veri tipi elde edilir. Bu aynı zamanda **long** veri tipidir. Ancak **long** veri tipini **long** sıfatı ile tanımladığımızda **long long** veri tipi elde edilir.

```
long int uzunTamSayi; /* 32 bit işlemcide 32 bit, 64 bit işlemcide 64 bit değer aralığı vardır. */
long long cokUzunTamSayi; /* 64 bitlik değer aralığı vardır */
```

`double` veri tipini `long` sıfatı ile tanımladığımızda `long double` veri tipi elde edilir. CLang/GCC ve Spark, PowerPC gibi bazı platformlarda `long double` veri tipi 16 bayt (128 bit) iken, diğer platformlarda 8 bayt (64 bit) olabilir. Bu nedenle, `sizeof(long double)` ifadesi platforma bağlı olarak farklı sonuçlar verebilir.

Değişkenin kapsamı (`variable scope`); Değişkenin bildiriminin (`variable declaration`) yapılabileceği, değişkene erişilebileceği (`access`), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır.

Değişken Kapsamı

Değişkenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

4.4 Değişken Kimliklendirme Kuralları

Kimliklendirme Kuralı

C programlama dilinde değişkenlerin kimliklendirilmesinde (`identifier definition`) dört temel kural vardır. Bunlar;

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. **Kimliklerin en fazla 31 karakter olması tercih edilir!.** Güncel C standartlarında (C99/C11) derleyicilerin iç (`internal`) kimlikler için en az 63, dış (`external`) bağlantı kimlikleri için en az 31 karakteri anlamlı kabul etmesi garanti edilir.
4. **Kimliklendirmede ancak İngiliz alfabesindeki Büyük ve Küçük harfler, rakamlar ile alt çizgi karakteri kullanılabilir.**

Aşağıda kimliklendirme kurallarına uygun tanımlama yapılmıştır;

```
/* Bu program değişken tanımlama örneklerini içerir. */
int main() {
    /* Tamsayı değişkenler: */
    char yas;
    int tam_sayi;
    long uzun_tam_sayi;
    /* Gerçek Sayı Değişkenler:*/
    float kilo;
    double reel_sayi;
    /*
    Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir.
    */
    int kat1,kat2,kat3;
    float olcek1,olcek2;
    return 0;
}
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığını artırmak için **deve yazımı** (`camel case`) stili kimlik verilir. Bu stilde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcılarının işini kolaylaştırmak için ahlaki olarak tercih edilir;

```
/* Bu program camelCase değişken kimliklendirme örneklerini içerir.*/
int main()
{
    int sayac;
    int ogrenciBoy;
    int asansorunOlduguKat;
    float asansorAirligi;
    int ogrenciYasi;
    char ilkHarf;
    char ogrenciAdi[50];
    char ogrenciSoyadi[50];
    float ortalamaNot;
    return 0;
}
```

4.5 Sabit Değişkenler

Programcı, kodlama yaparken bazı **sabitleri** (**constant**) ister istemez kullanır. Sabitler iki çeşittir. Bunların biri **değişmez** (**literal**), diğeri ise değeri değiştirilemeyen **sabit değişkenlerdir** (**const variable**). Derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısı olan **değişmezler** (**literal**) kullanılır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir.

Başlatma

Değişkenler tanımlanırken sahip olacağı ilk değerleri değişmezler ile verme işlemine **başlatma** (**initialization**) adı verilir.

Aşağıda çeşitli değişkenleri başlatma örneği verilmiştir;

```
1 /* Bu program değişkenlere ilk değer (initial value) vermede kullanılan için değişmez (literal)
   → örneklerini içerir. */
2 int main() {
3     int i,j=0;
4     char c=65;
5     float f=2.5f;
6     char harf='A'; //char harf=65;
7     return 0;
8 }
```

Yukarıda verilen kod şu şekilde açıklanabilir;

- ◇ 3. Satırdaki **int** i, j=0; talimatıyla, **int** veri tipindeki **i** değişkeni tanımlandı ve yine **int** veri tipindeki **j** kimlikli değişkene **0** tamsayı değişmezi (**integer literal**) ile ilk değer verildi. Burada sadece **j** değişkeni **0** ile **başlatılmıştır** (**initialize**).
- ◇ 4. Satırda **char** c=65; talimatıyla, **char** veri tipindeki **c** kimlikli deki değişkene **65** tamsayı değişmezi ile ilk değer verildi.
- ◇ 5. Satırda **float** f=2.5f; talimatıyla, **float** veri tipindeki **f** kimlikli değişkene Türkçe yazılışı 2,5 olan kayan noktalı değişmezi ile ilk değer verildi. Kod İngilizce yazıldığından, kodun içerisindeki gerçek sayı değişmezleri için **ondalık ayracı nokta olarak yazılır**.
- ◇ 6. Satırda **char** harf='A'; talimatıyla, **char** veri tipindeki **harf** kimlikli değişkene **'A'** karakter değişmezi (**character literal**) ile ilk değer verildi. **Karakter değişmezler tek tırnak içinde yazılır**. Klavyeden basılan her bir harf bellekte sayı olarak tutulur. **'A'** karakterinin kodu **65**, **'B'** karakterinin kodu **66**, ... Bu **ANSI** (**American National Standards Institute**) standardıdır. Bu nedenle bu talimat **char** harf=65; şekilde de yazılabilir. Günümüzde ANSI standardı yerine birden fazla karakterden oluşan **UTF-8** ve **UTF-16** karakter kodları kullanılmaktadır.

Boş Karakter

Kodu sıfır olan karakter ise **boş karakter** (**NULL Character**) olarak adlandırılır.

Aşağıdaki kod örneğinde programcı, sırasıyla **3**, **3.14f**, **2.0f** ve **3.14f** olmak üzere dört farklı değişmez (**literal**) kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir ve çalıştırma anında (**run time**) değiştirilemez.

```
/* Bu program, değişmez örneğidir. */
int main() {
    int yariCap=3;
    float daireninAlani=3.14f*yariCap*yariCap;
    float daireninCevresi=2.0f*3.14f*yariCap;
    return 0;
}
```

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir makine kodunu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const** sıfatıyla (**const qualifier**) **sabit değişkenler** (**const variable**) tanımlanabilir. Sabit değişkenler etik olarak büyük harfle kimlendirilir.

```
/* Bu program, const sıfatı örneğidir. */
int main() {
    int yariCap=3;
```



```

const float PI=3.14f; // Sabit değişken tanımı!
float daireninAlani=PI*yariCap*yariCap;
float daireninCevresi=2.0f*PI*yariCap;
return 0;
}

```

Böylece aynı sabit `PI` sabit değişkeni birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Tanımlama (**definition**) sırasında `const` sıfatı kullanılan değişkene, ilk değer (**initial value**) değişmez (**literal**) olarak verilmelidir. **Daha sonra bu değişkene bir değer ataması olamaz!**

Bir değişkeni sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- ♦ **Gelişmiş Kod Okunabilirliği:** Bir değişkene `const` sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- ♦ **Gelişmiş Veri Tipi Güvenliği:** `const` kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- ♦ **Gelişmiş Optimizasyon:** Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için `const` değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- ♦ **Daha İyi Bellek Kullanımı:** Değişkenlere `const` sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- ♦ **Gelişmiş Uyumluluk:** Değişkenlere `const` sıfatı vermek, kodunuzu `const` değişkenleri kullanan diğer başlıklarla daha uyumlu hale getirebilir.
- ♦ **Gelişmiş Güvenilirlik:** `const` kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.6 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında yazdığımız talimatları (**statement**) sırasıyla çalıştırır. C dilinde her talimat noktalı virgül ile bittiği 4.2.2 başlığında anlatılmıştı. Aşağıdaki örnek programı göz önüne alalım;

```

1 int main() {
2     float boy=1.80f;
3     float kilo=100.0f;
4     float bki;
5     float boyKare;
6     boyKare=boy*boy;
7     bki=kilo/boyKare;
8     boy=1.50f;
9     return 0;
10 }

```

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, `main()` fonksiyonu içindeki ilk talimatla başlar ve aşağıdaki sırayla icra edilir;

1. İlk olarak 2. satırdaki `float boy=1.80f;` talimatıdır: `boy` Değişkenine bellek tahsis edilerek değişkene `1.80f` değeri atanır. Yani `boy` değişkeni `1.80f` değeriyle **başlatılır** (**initialization**).
2. Sonra 3. satırdaki `float kilo=100.0f` talimatı icra edilir: `kilo` Değişkeni `100.0f` ile başlatılır.
3. Daha sonra 4. satırdaki `float bki;` talimatı icra edilir: tanımlanan `bki` değişkenine sadece bellek tahsis edilir ve değeri belli değildir. Bazı derleyiciler değerleri sıfır olarak varsayar.
4. Daha sonra 5. satırdaki `float boyKare;` talimatı icra edilir: Tanımlanan `boyKare` değişkenine sadece bellek tahsis edilir ve değeri belli değildir.
5. Daha sonra 6. satırdaki `boyKare=boy*boy;` icra edilir: Önceden tanımlanmış `boyKare` değişkenine `boy` değişkeni kendisiyle çarpılarak işlem sonucu atanır. `boyKare` değişkeni öncesinde tanımlanmamış olsaydı derleme hatası alacaktık.
6. Daha sonra 7. satırdaki `bki=kilo/boyKare;` talimatı icra edilir: `kilo` Değişkenini `boyKare` değişkenine böler ve çıkan sonucu `bki` değişkenine atar.
7. Daha sonra 8. satırdaki `boy=1.50f` talimatı icra edilir: `boy` Değişkenine `1.50f` atanarak değeri değiştirilir.
8. Son olarak 9. satırdaki `return 0;` talimatı icra edilir: `return` Talimatı içinde bulunan fonksiyondan çıkış talimatıdır. Ana fonksiyondan çıkıldığı için işletim sistemine dönlür.

4.7 İşleçler

İşleçler (**operator**) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

Atama İşleci

En çok kullanılan işleç, atama (=) işlecidir. **Atama işleci, her zaman sağdaki işleneni soldaki değişkene atar.** Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

§4.7.1 Aritmetik İşleçler

C programlama dilinde aritmetik işleçlerin çalışma şekli **işlenenlerin** (**operand**) veri tipine göre değişir. Aritmetik işlemler ve atama işleci için kurallar;

1. **İşlenenlerden birinin bellekte kapladığı yer küçük ise ikisi aynı olacak şekilde bellekte daha fazla yer kaplayanına dönüştürülür** ve işlem sonra yapılır ve sonuç dönüştürülen veri tipinde üretilir. Örneğin **char** veri tipindeki değişken ile **long** veri tipindeki bir değişken toplanmaya çalışıldığı zaman **char** veri tipindeki değişkenin değeri **long** veri tipine otomatik dönüştürülür. Sonuç **long** veri tipinde üretilir.
2. **İşlenenlerden biri tamsayı diğeri kayan noktalı sayı ise işlem yapılmadan önce işlenenler kayan noktalı sayıya dönüştürülür.** Örneğin **int** veri tipindeki değişken ile **float** veri tipindeki bir değişken toplanmaya çalışıldığı zaman **int** veri tipindeki değişkenin değeri **float** veri tipine otomatik dönüştürülür. Sonuç **float** veri tipinde üretilir.

Aritmetik işleçlerle (**arithmetic operator**), temel dört işlem ile modül işlemleri yapılır.

İşleç	Adı	Açıklama
+	Toplama	İki işleneni toplar.
-	Çıkarma	Soldaki işlenenden sağdakini çıkarır.
*	Çarpma	İki işleneni çarpar.
/	Bölme	Soldaki işleneni sağdakine böler. Her iki işlenen tam sayı ise sonuç tam sayıdır (ondalık kısım atılır).
%	Modül	Soldaki işlenenin sağdakine bölümünden kalan değeri verir.

Tablo 4.2: Aritmetik İşleçler

Bu işleçlere ilişkin örneği aşağıdaki görebilirsiniz;

```
/* Bu program, aritmetik işleç örneğidir. */
int main() {
    int a = 9, b = 4, res;
    float fa=6.6, fb= 2.2, fres;
    res = a + b; // işlem sonucu res=13
    res = a - b; // işlem sonucu res=5
    res = a * b; // işlem sonucu res=36
    res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
    /* Bölme işlecinin işlenenleri tamsayı olduğunda bölme sonucundaki tamsayı olarak hesaplanır.
    ↳ Bölme işleminin tam kısmını içeren sonuç döndürülür. */
    res = a / 4.3 ;
    /* işlem sonucu res tamsayı olduğundan; res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */

    res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
    fres= fa / fb; // işlem sonucu res=6.6/2.2=3.00
    return 0;
}
```

C ve C++ programlama dilinde tam sayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür. Bu nedenle diğer programlama dillerinden ayrılır. Aşağıda buna ilişkin örnek verilmiştir.

```
/* Bu program, aritmetik işleç BÖLME örneğidir. */
int main () {
```



```

int a = 19 / 2 ; // a = 2*9+1 = 9
int b = 18 / 2 ; // b = 9
int c = 255 / 4; // c = 4*63+3 = 63
int d = 45 / 4 ; // d = 4*11+1 = 11

double aa = 19 / 2.0 ; // aa = 9.5
double bb = 18.0 / 2 ; // bb = 9.0
double cc = 255 / 2.0; // cc = 127.5
double dd = 45.0 / 4 ; // dd = 11.25

double ee = 45 / 4 ;
/* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
return 0;
}

```

§4.7.2 Tekli İşleçler

Tekli işleçler (**unary operator**) sadece tek bir işlenen üzerinde işlem yapar.

İşleç	Adı	Açıklama
+	Tekli Artı	İşlenenin işaretini pozitif yapar (genellikle varsayılan durumdur).
-	Tekli Eksi	İşlenenin işaretini tersine çevirir (x ise $-x$, $-x$ ise x yapar).
++	Arttırım	İşlenenin değerini 1 artırır. Önek (prefix) ise önce artırır sonra değeri döndürür. Sonek (postfix) ise önce değeri döndürür sonra artırır.
--	Eksiltme	İşlenenin değerini 1 eksiltir. Önek ve sonek kullanım kuralları artırım işleci ile aynıdır.
!	Mantıksal Değil	Bool değerini tersini alır (Sıfırdan farklı ise 0, 0 ise 1 yapar).
(type)	Tip Dönüşümü	İşleneni parantez içinde belirtilen veri tipine (casting) dönüştürür.
&	Adres Operatörü	Değişkenin bellek adresini döndürür.
*	Başvuru Kaldırma	Göstericinin (pointer) işaret ettiği değeri döndürür.
sizeof	Bellek Boyutu	Temel veri tiplerinin bellek boyutunu geri döndürür.

Tablo 4.3: Tekli İşleçler

Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```

/* Bu program, tekli işleç örneğidir. */
int main() {
    int a = 5, b = 5, res;
    res = -a; // tekli eksisi (unary minus): res=-5;
    res = ++a+2;
    /* ++a işlemini önce-arttırım (pre-increment) olarak adlandırılır. a değişkeni işleme girmeden
       → önce artırılır.
       İşlem sonucu res=8, a=6 */
    res = 2+b++;
    /* b++ işlemini sonra-arttırım (post-Increment) olarak adlandırılır. b değişkeni işleme
       → girdikten sonra artırılır.
       İşlem sonucu res=7, b=6 */
    a=5; b=5;
    /* iki talimat(statement) yan yana yazılmıştır.Daha fazla da yazılabilir. */
    res = --a+2;
    /* Pre-Decrement: işlemden sonra res=6, a=4 */
    res = 2+b--;
    /* Post-Decrement: işlemden sonra res=7, b=4 */

    char c=(char)a; //int veri tipinden char veri tipine dönüşüm.
    int x = (int)3.14; //double veri tipinden int veri tipine dönüşüm.

    a=1; b=0;
    res=!a; // işlem sonucu res=0
    res=!b; // işlem sonucu res=1

    res=sizeof(char); /* char veri tipinin bellek miktarını öğrenme: işlem sonucu res=1 */
    res=sizeof a; /* a değişkeninin bellek miktarını öğrenme: işlem sonucu res=4 */
}

```

```
return 0;
}
```

§4.7.3 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler.

Bilgi

C dilinde, doğru (**true**) ya da yanlış (**false**) gibi değer tutabilen mantıksal veri tipi (**data type**) bulunmaz. Bunun yerine tam sayılar, **mantıksal veri tipi** olarak kullanılır. Tam sayının değeri 0 ise **YANLIŞ**, sıfırdan farklı ise **DOĞRU** olarak anlamlandırılır.

İşleç	Adı	Açıklama
==	Eşittir	Sol ve sağdaki işlenenler birbirine eşitse 1 döner.
!=	Eşit Değildir	İşlenenler birbirine eşit değilse 1 döner.
>	Büyüktür	Soldaki işlenen sağdakinden büyükse 1 döner.
<	Küçüktür	Soldaki işlenen sağdakinden küçükse 1 döner.
>=	Büyük veya Eşit	Soldaki işlenen sağdakinden büyük veya ona eşitse 1 döner.
<=	Küçük veya Eşit	Soldaki işlenen sağdakinden küçük veya ona eşitse 1 döner.

Tablo 4.4: İlişkisel İşleçler ve Mantıksal Karşılıkları

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, ilişkisel işleç örneğidir. */
int main() {
    int a = 5, b = 6, res;

    res = a<b; // işlem sonucu res=1
    res = a<=b; // işlem sonucu res=1
    res = a>b; // işlem sonucu res=0
    res = a>=b; // işlem sonucu res=0
    res = a==b; // işlem sonucu res=0
    res = a!=b; // işlem sonucu res=1
    return 0;
}
```

§4.7.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operator**) özellikle ikilik tabandaki tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir. Aslında bu işlemler çarpma, toplama, bölme ve çıkarma işlemleri için kullanılır.

İşleç	Adı	Açıklama
&	VE (AND)	Karşılıklı iki bit de 1 ise sonuç 1, diğer durumlarda 0 olur.
	VEYA (OR)	Karşılıklı bitlerden en az biri 1 ise sonuç 1, ikisi de 0 ise 0 olur.
^	ÖZEL VEYA (XOR)	Bitler birbirinden farklıysa 1, aynıysa 0 olur.
~	DEĞİL (NOT)	Tüm bitleri tersine çevirir (0 → 1, 1 → 0).
<<	Sola Kaydırma	Bitleri belirtilen sayı kadar sola kaydırır. Sağdan 0 ile doldurur.
>>	Sağa Kaydırma	Bitleri belirtilen sayı kadar sağa kaydırır.

Tablo 4.5: Bit Düzeyi İşleçler

Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
/* Bu program, bit düzeyi işleç örneğidir. */
int main() {
    // C99 ve sonrasında binary gösterimi için '0b' öneki kullanılır.
    char a = 0b00000110; // Onluk tabanda: 6
    char b = 0b00000010; // Onluk tabanda: 2

    char bitwise_and = a & b; // 00000010 (Onluk: 2)
    char bitwise_or = a | b; // 00000110 (Onluk: 6)
    char bitwise_xor = a ^ b; // 00000100 (Onluk: 4)
    char bitwise_not = ~b; // 11111101 (Onluk: -3)
}
```

```

char left_shift = b << 2; // 00001000 (Onluk: 8)
char right_shift = b >> 1; // 00000001 (Onluk: 1)

return 0;
}

```

§4.7.5 Mantıksal İşleçler

Mantıksal ifadeler doğru (**true**) ve yanlış (**false**) olabilen ifadelerdir. C dilinde mantıksal bir veri tipi yoktur ve int veri tipi bunun için kullanılır. Sıfırdan farklı bir tamsayı ifade doğru, sıfır olan tamsayı yanlış olarak değerlendirilir. C Dilinde VE(AND), VEYA(OR) ve DEĞİL (NOT) gibi mantıksal ifadelerin işlenmesi için aşağıdaki işlemler kullanılır. Mantıksal işlemlere (**logical operator**) işlenen olarak giren ifadeler öncelikle **int** veri tipine dönüştürülür. Yani bu işlemler için beklenen işlenen (**operand**) bir tamsayı ifadedir.

İşleç	Mantıksal İşlem ve Kısa Devre Davranışı
&&	Koşullu Mantıksal VE: Her iki işlenen de 1 ise 1 döner. Soldaki işlenen 0 ise sağdaki ifade değerlendirilmez (Kısa Devre).
	Koşullu Mantıksal VEYA: İşlenenlerden en az biri 1 ise 1 döner. Soldaki işlenen 1 ise sağdaki ifade değerlendirilmez (Kısa Devre).
!	Mantıksal DEĞİL: İşlenenin mantıksal durumunu tersine çevirir. 1 değerini 0, 0 değerini 1 yapar.

Tablo 4.6: Mantıksal İşlemler ve Kısa Devre Özelliği

Aşağıda verilen ilişkisel ve mantıksal işlemlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```

/* Bu program, mantıksal işlemler örneğidir. */
int main() {
    int a = 5, b = 6;
    int res;
    res = a&&b; // işlem sonucu res=1
    res = a||b; // işlem sonucu res=1
    res = !a;   // işlem sonucu res=0
    return 0;
}

```

Bu işlemlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin bir kısmı işlenmediğinden, **koşullu olarak çalışırlar**. Buna **kısa devre** adı verilir. Derleyici, sonucun kesinleştiği noktada işlem yapmayı bırakır. Örneğin (a == 5 && b == 10) ifadesinde **a** değeri 5 değilse, **b**'nin ne olduğuna bakmaya gerek duymaz. Bu durum sadece hız kazandırmaz, aynı zamanda olası hataları da önler:

```

int a=10, b=10;
if (a == 5 && b == 10) // a 5 den farklı olduğu için b==10 kontrol edilmez!
{
    // Buradaki işlemler yapılmaz!...
}

```

C dilinde tekli **&** ve **|** işlemleri de mantıksal ifadelerle kullanılabilir. Ancak bunlar **kısa devre yapmazlar**; yani sol taraf ne olursa olsun sağ tarafı da kontrol ederler.

§4.7.6 Atama İşlemleri

Atama işlemleri (**assignment operator**) en çok kullanılan işlemlerdir. **Atama işlemleri her zaman sağdaki değeri sola atar!** Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (**variable**) olmak zorundadır.

Aşağıda verilen atama işlemlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```

/* Bu program, atama işlemleri örneğidir. */
int main() {
    int a = 10; // a değişkenine 10 değeri = işleciyle atanıyor.
    a += 10;    // a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor.
    a -= 10;    // a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor.
    a *= 10;    // a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.
    a /= 10;    // a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.
    a %= 10;    // a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup ona eşitleniyor.
    return 0;
}

```

İşleç	Adı	Örnek	Açıklama
=	Temel Atama	x = 5	Sağdaki değeri soldaki değişkene aktarır.
+=	Toplayarak Atama	x += 3	x = x + 3 ile aynıdır.
-=	Çıkararak Atama	x -= 2	x = x - 2 ile aynıdır.
*=	Çarparak Atama	x *= 4	x = x * 4 ile aynıdır.
/=	Bölerek Atama	x /= 2	x = x / 2 ile aynıdır.
%=	Modül Alarak Atama	x %= 3	x = x % 3 ile aynıdır.
&=	Bit Düzeyi VE ile Atama	x &= 1	x = x & 1 ile aynıdır.
=	Bit Düzeyi VEYA ile Atama	x = 1	x = x 1 ile aynıdır.
^=	Bit Düzeyi ÖZEL VEYA ile Atama	x ^= 1	x = x ^ 1 ile aynıdır.
<<=	Bit Düzeyi Sola Kaydırma ile Atama	x <<= 1	x = x << 1 ile aynıdır.
>>=	Bit Düzeyi Sağa Kaydırma ile Atama	x >>= 1	x = x >> 1 ile aynıdır.

Tablo 4.7: Atama İşleçleri

}

§4.7.7 Virgül İşleci

C dilinde virgül işleci (**comma operator**) iki farklı amaca hizmet eder: ayraç (**separator**) olarak ve bir ikili (**binary**) işleç olarak.

Virgül İşleci

Virgül bir işleç olarak kullanıldığında, birden fazla ifadeyi soldan sağa doğru sırayla çalıştırır. Bütün ifadeleri baktıktan sonra en sağdaki ifadenin değerini ve türünü sonuç olarak döndürür.

```
#include <stdio.h>
int main() {
    int a;

    int b = (a = 5, a + 10);
    // 1. a = 5 çalışır
    // 2. a + 10 hesaplanır
    // 3. En sağdaki değer (15) b değişkenine atanır

    printf("a: %d, b: %d\n", a, b); // a: 5, b: 15
    return 0;
}
```

Bu işleç, **for** döngülerinde birden fazla sayacı güncellemek için çokça kullanılır.

Ayraç

Her virgül bir operatör değildir. Değişken tanımlarında veya fonksiyon parametrelerinde kullanılan virgüller ayraçtır (**separator**):

- ◊ **int x = 5, y = 10;** talimatında virgül bir operatör değil, değişkenleri ayıran bir simgedir. Değer döndürmez veya soldan sağa işlem sırası garanti etmez.
- ◊ **fonksiyon(a, b);** talimatıyla çağrılan fonksiyon argümanları arasındaki virgül ayraçtır. Argümanların hangi sırayla çalıştırılacağı ileride işlenecektir.

§4.7.8 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, işleç öncelikleri örneğidir. */
int main() {
    int a=10, b=5;
    a = a-b+2; // Öncelik Sırası +, -, = İşlem sonunda a=7
    a = a/b+2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
    a = 2*10-20/2+3+(12-10); // Öncelik Sırası: (-) * / - + +
    /* İfadenin çözüm sırası:
```

Öncelik	Kategori	İşleçler	Birleşme
1	Temel (postfix)	f(), [], --, ++, (type){list}	Soldan Sağa
2	Tekli (unary)	++, --, ~, !, (type), *, &, sizeof, _Alignof	Sağdan Sola
3	Çarpma/Bölme	*, /, %	Soldan Sağa
4	Toplama/Çıkarma	+, -	Soldan Sağa
5	Bit Kaydırma	<<, >>	Soldan Sağa
6	İlişkisel	<, <=, >, >=	Soldan Sağa
7	Eşitlik	==, !=	Soldan Sağa
8	Bit Düzeyi VE	&	Soldan Sağa
9	Bit Düzeyi XOR	^	Soldan Sağa
10	Bit Düzeyi VEYA		Soldan Sağa
11	Mantıksal VE	&&	Soldan Sağa
12	Mantıksal VEYA		Soldan Sağa
13	Üçlü Koşul (ternary)	?:	Sağdan Sola
14	Atama	=, +=, -=, *=, /=, %%, <<=, >>=, &=, ^=, =	Sağdan Sola
15	Virgül	,	Soldan Sağa

Tablo 4.8: C Dili İşleç Öncelik Hiyerarşisi (Düzeltilmiş)

```

> 2*10-20/2+3+2
> 20-20/2+3+2
> 20-10+3+2
> 10+3+2
> 13+2
> 15 */
return 0;
}

```

4.8 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Bu duruma aşağıdaki klasik örnek verilir;

```

#include <stdio.h>
int main()
{
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    if(a + b == c)
        printf("Bu satır İcra Edilmez!\n"); //IEEE754 standardında işlem yaparsak.
    else
        printf("Kayan Noktalı Sayılar İkili Sayı Sisteminde Tutulduğundan Yaklaşık Olarak
        ↪ Hesaplanır." );
    return 0;
}

```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.333333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir).

4.9 Düşün ve Çöz

- ♦ **Düşün:** C dilindeki anahtar kelimelerin (**keywords**) niçin değişken ismi olarak kullanılamayacağını dilin söz dizimi kuralları açısından değerlendiriniz.
- ♦ **Çöz:** `int` 1sayı; `float` ortalama_deger; `char` switch; ve `double` _toplam; değişken tanımlamalardan hangileri geçerlidir, hangileri geçersizdir? Geçersiz olanların sebebini açıklayınız.
- ♦ **Çöz:** İşlem önceliklerini dikkate alarak `int` sonuc = 15 + 6 / 2 * 3 - (8 % 3); işleminin sonucunu adım adım hesaplayınız.
- ♦ **Çöz:** a = 5, b = 10, c = 15 değerleri için a + b * c / a - b % 3 ifadesinin sonucunu işlem öncelik sırasına göre adım adım hesaplayan bir program parçacığı yazınız.

- ◇ **Düşün:** `x = x + 1;` ile `x++;` ifadeleri arasında performans açısından (assembly seviyesinde) bir fark var mıdır? Araştırınız.
- ◇ **Çöz:** Bit düzeyi (**bitwise**) işlemleri kullanarak, bir sayının 2'nin tam kuvveti olup olmadığını bulan bir C ifadesi yazınız.
- ◇ **Düşün:** Mantıksal VE (`&&`) operatöründe kısa devre (**short-circuit**) değerlendirmesi nedir? İlk koşul yanlış olduğunda ikincisine bakılmaması neden önemlidir?
- ◇ **Çöz:** Virgül operatörünün (`a = (b=3, b+2);`) çalışma mantığını analiz ediniz ve a ile b'nin son değerlerini tahmin ediniz.
- ◇ **Çöz:** Aşağıdaki kod bloğu çalıştırıldığında ekrana ne basılır? Değişkenlerin değerlerindeki değişimleri adım adım takip ediniz.

```
int a = 5, b;
b = a++;
printf("a: %d, b: %d\n", a, b);
b = ++a;
printf("a: %d, b: %d\n", a, b);
```
- ◇ **Çöz:** Aşağıdaki kod parçası yürütüldüğünde y ve z değişkenlerinin son değeri ne olur? Kısa devre (**short-circuit**) özelliğini dikkate alarak açıklayınız.

```
int x = 0, y = 10, z = 20;
if (x != 0 && (y = z + 5)) {
    z++;
}
```
- ◇ **Çöz:** `a = 12 (00001100)` ve `b = 25 (00011001)` tam sayıları veriliyor. `a & b`, `a | b`, `a ^ b` ve `a << 2` bit düzeyindeki işlemlerin sonuçlarını ikili (**binary**) ve onlu (**decimal**) sistemde hesaplayınız
- ◇ **Çöz:** Kullanıcıdan alınan 8-bitlik pozitif bir tam sayının en sağdaki (Least Significant Bit - LSB) 3. bitini duruma bakılmaksızın 1 yapan ve sonucu ekrana hex (0x..) olarak basan C kodunu yazınız.
- ◇ **Çöz:** C dilinde tamsayı bölmesi ile kayan noktalı sayı bölmesi farkını gösteren **float** `f1 = 7 / 2;`, **float** `f2 = (float)7 / 2;` ve **float** `f3 = 7.0 / 2;` ifadelerin çıktılarını sebebiyle açıklayınız.
- ◇ **Çöz:** `const int x = 10;` ifadesi ile `#define X 10` makro tanımı arasındaki farkları bellek kullanımı ve derleme aşaması yönünden karşılaştırınız.
- ◇ **Çöz:** Aşağıdaki kodun çıktısı ne olur?

```
int a = 2, b = 3, c = 4;
int sonuc = a + b * c / a % b - (a > b ? a : b);
printf("%d\n", sonuc);
```
- ◇ **Çöz:** Bir sayının 2'nin kuvveti (1, 2, 4, 8, 16...) olup olmadığını bit düzeyindeki işlemleri (`&`, `~`, vb.) kullanarak tek bir satırda kontrol eden mantıksal ifadeyi yazınız.